

BCC202 - Estruturas de Dados I

Aula 19: Ordenação: *Heap Sort*

Pedro Silva

Universidade Federal de Ouro Preto, UFOP
Departamento de Computação, DECOM
Email: silvap@ufop.edu.br



Conteúdo

Operações usando *Heap*

Inserção

Remoção

Ordenação usando *Heap*

Heap Sort

Animações

Implementação

Análise do método

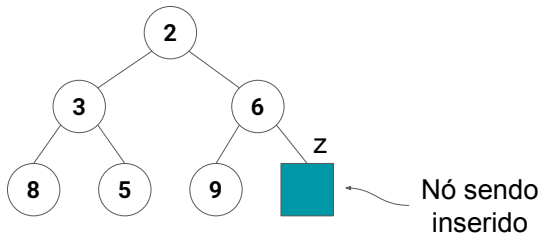
Considerações Finais

Bibliografia

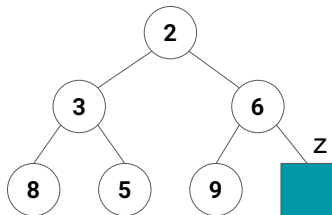
Exercício

Operações usando *Heap*

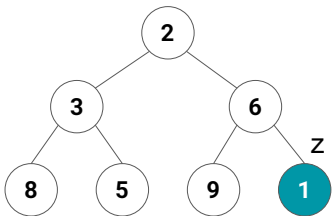
Inserir um Nó no Heap



Inserir um Nó no Heap

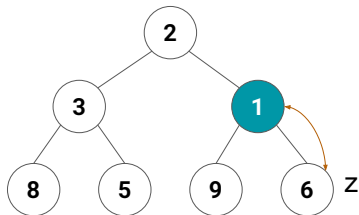


Nó sendo
inserido

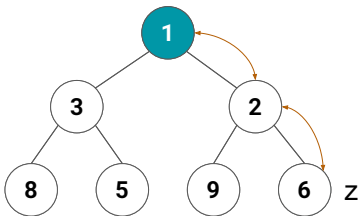
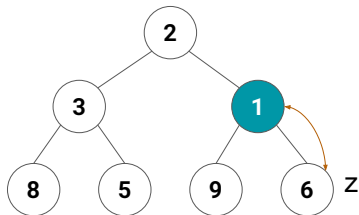


Comparar o nó inserido com os pais e trocar enquanto ele for menor que o pai ou até que ele seja o nó raiz

Inserir um Nó no Heap

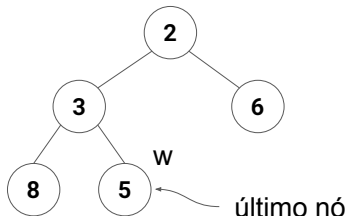


Inserir um Nó no Heap

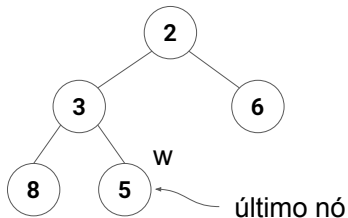


Na pior das hipóteses o custo de uma inserção será $O(\log n)$, equivalente à altura da árvore

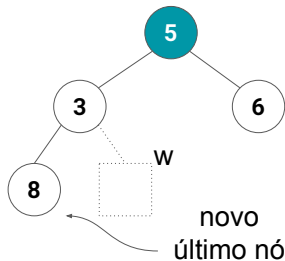
Remover um Nó do Heap



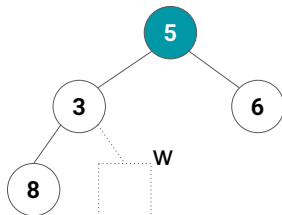
Remover um Nó do Heap



Trocar o nó raiz pelo
último nó do heap e
remover o último nó

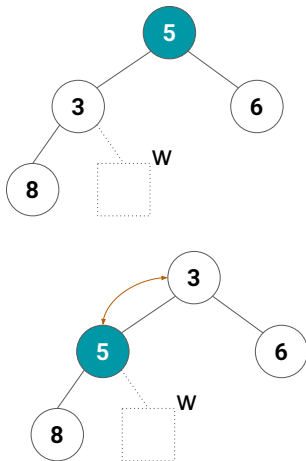


Remover um Nó do Heap



Refazer o heap!

Remover um Nó do Heap



Refazer o heap!

Na pior das hipóteses o custo de uma remoção será $O(\log n)$, equivalente à altura da árvore

Ordenação usando *Heap*

Ordenação usando *Heap*

- ▶ As operações das filas de prioridades podem ser utilizadas para implementar algoritmos de ordenação.
- ▶ Basta utilizar repetidamente a operação **INSERE** para construir a fila de prioridades.
- ▶ Em seguida, utilizar repetidamente a operação **RETIRA** para receber os itens na ordem correta.

Listas vs. Métodos de Ordenação

- ▶ O uso de listas lineares não ordenadas corresponde ao método _____ .
- ▶ O uso de listas lineares ordenadas corresponde ao método _____ .
- ▶ O uso de heaps corresponde ao método _____ .

Listas vs. Métodos de Ordenação

- ▶ O uso de listas lineares não ordenadas corresponde ao método **Selection Sort**.
- ▶ O uso de listas lineares ordenadas corresponde ao método _____ .
- ▶ O uso de heaps corresponde ao método _____ .

Listas vs. Métodos de Ordenação

- ▶ O uso de listas lineares não ordenadas corresponde ao método **Selection Sort**.
- ▶ O uso de listas lineares ordenadas corresponde ao método **Insertion Sort**.
- ▶ O uso de heaps corresponde ao método _____ .

Listas vs. Métodos de Ordenação

- ▶ O uso de listas lineares não ordenadas corresponde ao método **Selection Sort**.
- ▶ O uso de listas lineares ordenadas corresponde ao método **Insertion Sort**.
- ▶ O uso de heaps corresponde ao método **Heap Sort**.

Heap Sort

Heap Sort

- ▶ Para reduzir o número de operações, será utilizado um Heap em que o **menor** valor é armazenado na **raiz**, ou seja, a **maior prioridade** é o item com a **menor** chave.
 - ▶ Teremos que o valor de um nó pai deverá ser sempre menor ou igual aos valores dos seus filhos.
- ▶ Utilizaremos repetidamente a operação **INSERE** para construir a fila de prioridades.
- ▶ Em seguida, utilizaremos repetidamente a operação **RETIRA** para receber os itens na ordem **inversa**.

Heap Sort

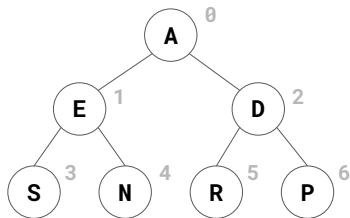
Algoritmo:

1. Construir o heap.
2. Troque o item na posição 0 do vetor (raiz do *heap*) com o item da posição $n - 1$.
3. Use o procedimento **REFAZ** para reconstituir o *heap* para os itens :
 $A[0], A[1], \dots, A[n - 2]$.
4. Repita os passos 2 e 3 com os $n - 1$ itens restantes, depois com os $n - 2$, até que reste apenas um item.

Exemplo de execução

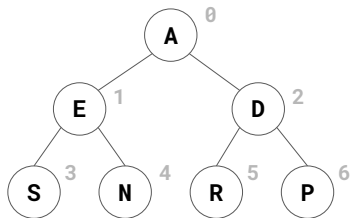
0	1	2	3	4	5	6
A	E	D	S	N	R	P

Exemplo de execução



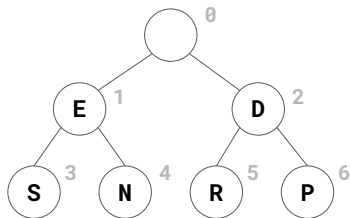
Exemplo de execução

Remover Raiz



Exemplo de execução

Remover Raiz

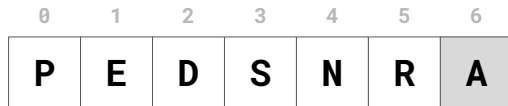
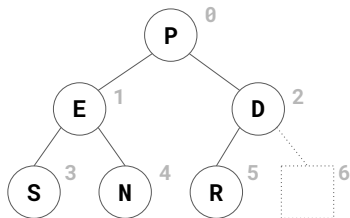


Item removido: A



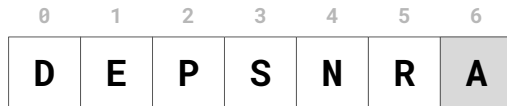
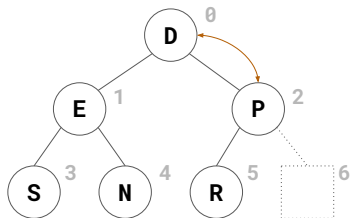
Exemplo de execução

Colocar na raiz
o último item



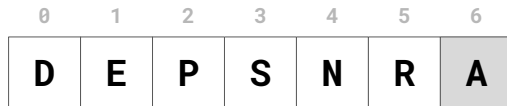
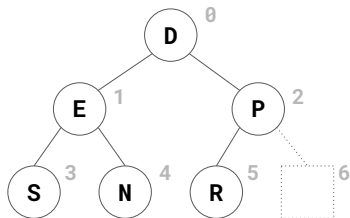
Exemplo de execução

Refazer o Heap



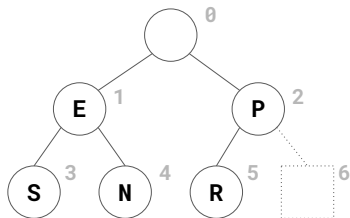
Exemplo de execução

Remover Raiz

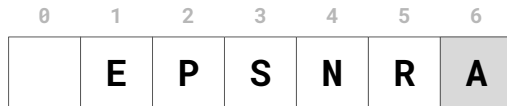


Exemplo de execução

Remover Raiz

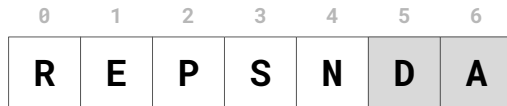
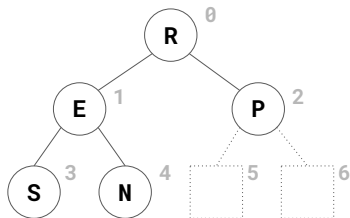


Item removido: D



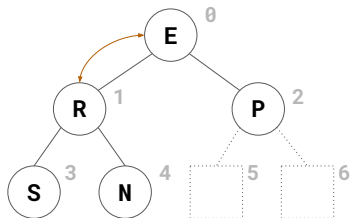
Exemplo de execução

Colocar na raiz
o último item



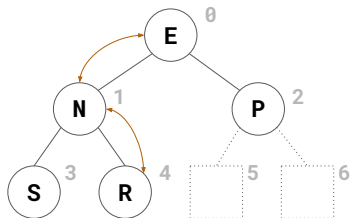
Exemplo de execução

Refazer o Heap



Exemplo de execução

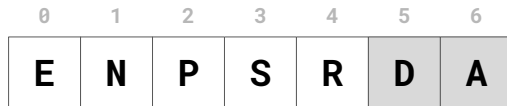
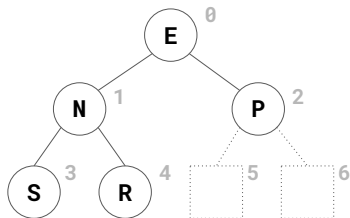
Refazer o Heap



0	1	2	3	4	5	6
E	N	P	S	R	D	A

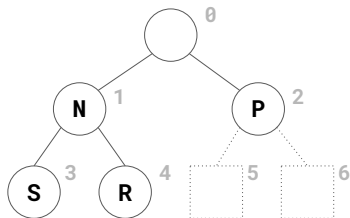
Exemplo de execução

Remover Raiz



Exemplo de execução

Remover Raiz

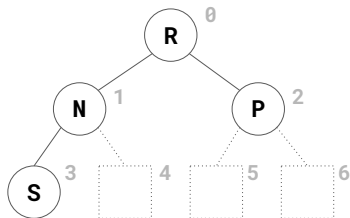


Item removido: E



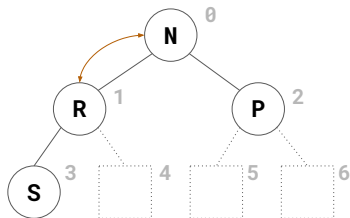
Exemplo de execução

Colocar na raiz
o último item



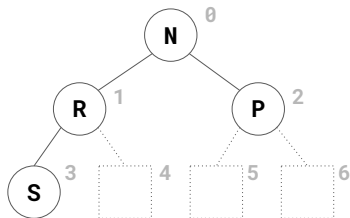
Exemplo de execução

Refazer o Heap



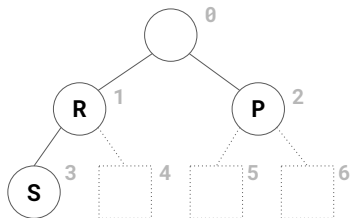
Exemplo de execução

Remover Raiz



Exemplo de execução

Remover Raiz

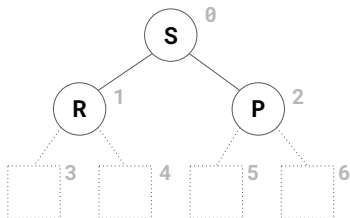


Item removido: N



Exemplo de execução

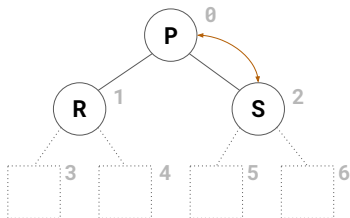
Colocar na raiz
o último item



0	1	2	3	4	5	6
S	R	P	N	E	D	A

Exemplo de execução

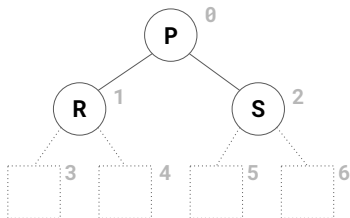
Refazer o Heap



0	1	2	3	4	5	6
P	R	S	N	E	D	A

Exemplo de execução

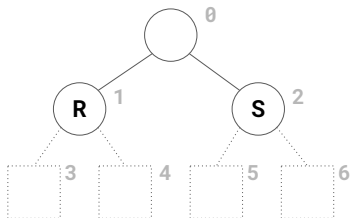
Remover Raiz



0	1	2	3	4	5	6
P	R	S	N	E	D	A

Exemplo de execução

Remover Raiz

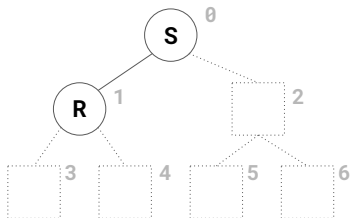


Item removido: P



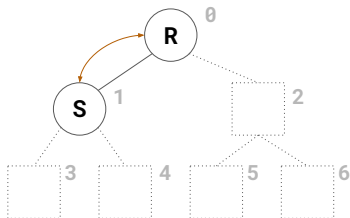
Exemplo de execução

Colocar na raiz
o último item



Exemplo de execução

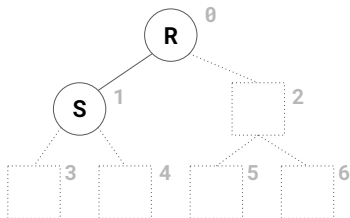
Refazer o Heap



0	1	2	3	4	5	6
R	S	P	N	E	D	A

Exemplo de execução

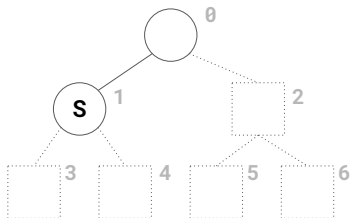
Remover Raiz



0	1	2	3	4	5	6
R	S	P	N	E	D	A

Exemplo de execução

Remover Raiz

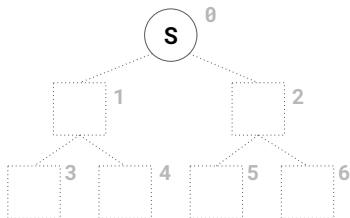


Item removido: R

0	1	2	3	4	5	6
	S	P	N	E	D	A

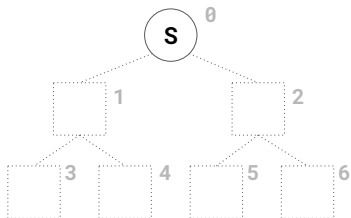
Exemplo de execução

Colocar na raiz
o último item



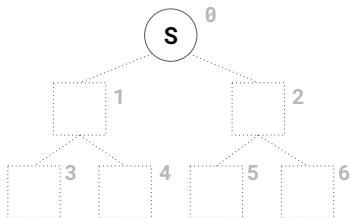
Exemplo de execução

Refazer o Heap



Exemplo de execução

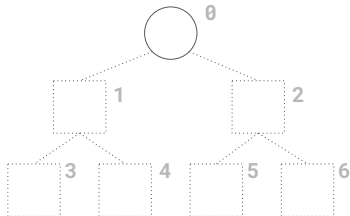
Remover Raiz



0	1	2	3	4	5	6
S	R	P	N	E	D	A

Exemplo de execução

Remover Raiz

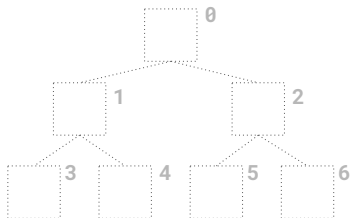


Item removido: S



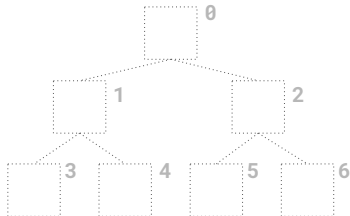
Exemplo de execução

Colocar na raiz
o último item



Exemplo de execução

Heap vazio!!!



Ordenação finalizada.



Vetor na ordem inversa!

A seguir, mais detalhes sobre o passo a passo do *Heap Sort*.



Animação



Vídeo

1 Algorithm: HEAP_REFAZ**Input:** int* v, int esq, int dir**2 begin****3** *i* ← esq**4** *j* ← *i* * 2 + 1**5** aux ← v[*i*]**6** **while** *j* ≤ dir **do****7** **if** *j* < dir **and** v[*j*] < v[*j* + 1] **then****8** *j* ← *j* + 1**9** **end****10** **if** aux ≥ v[*j*] **then****11** break**12** **end****13** v[*i*] ← v[*j*]**14** *i* ← *j***15** *j* ← *i* * 2 + 1**16** **end****17** v[*i*] ← aux**18 end**

// j recebe o outro filho de i

1 Algorithm: HEAP_CONSTROI**Input:** int* v, int n**2 begin****3** $esq \leftarrow (n/2) - 1$ *// esq = primeiro no antes do no folha do heap***4** **while** $esq \geq 0$ **do****5** HEAP_REFAZ(v, esq, n - 1)**6** $esq \leftarrow esq - 1$ **7** **end****8 end**

Implementação

1 **Algorithm:** HEAP_SORT

Input: int* v, int n

2 **begin**

3 HEAP_CONSTROI(v, n)

4 **while** $n > 1$ **do**

5 aux $\leftarrow v[n - 1]$

6 $v[n - 1] \leftarrow v[0]$

7 $v[0] \leftarrow aux$

8 $n \leftarrow n - 1$

9 HEAP_REFAZ(v, 0, $n - 1$)

// refaz o heap

10 **end**

11 **end**

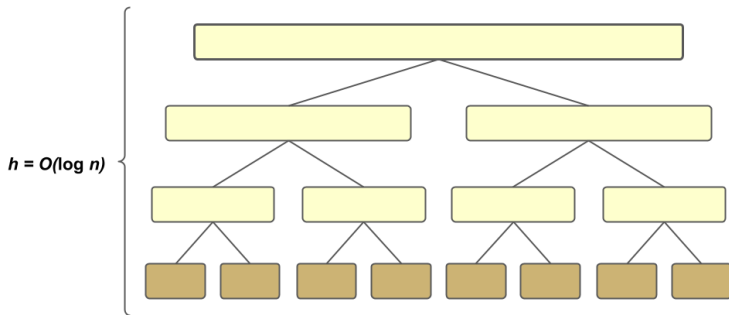
Análise do método

Análise do método

- ▶ O procedimento *Refaz* gasta cerca de $O(\log n)$ operações, no pior caso.
- ▶ Constrói – executa $O(n)$ x **Refaz**.
- ▶ *Loop* interno – executa $O(n)$ x **Refaz**.
- ▶ Logo, **Heap Sort** gasta um tempo de execução $O(n \log n)$ no pior caso.

Análise do método

- ▶ A altura h da árvore de execução é $O(\log n)$.
 - ▶ Refazer o heap, na pior das hipóteses, tem custo igual à altura da árvore.
- ▶ Construir o heap custa $O(n \log n)$.
- ▶ Logo: algoritmo é $O(n \log n)$.



Vantagens:

- ▶ O comportamento do *Heap Sort* é sempre $O(n \log n)$, qualquer que seja a entrada.

Desvantagens:

- ▶ O anel interno do algoritmo é bastante complexo se comparado com o do *Quick Sort*.
- ▶ O *Heap Sort* não é estável.

Considerações Finais

Conclusão

O *Heap Sort* é recomendado:

- ▶ Para aplicações que não podem tolerar eventualmente um caso desfavorável.
- ▶ Não é recomendado para arquivos com poucos registros, por causa do tempo necessário para construir o *heap*.
- ▶ A **ordenação por *heap*** introduz outra técnica de projeto de algoritmos: a utilização de uma estrutura de dados, nesse caso uma estrutura que denominamos *heap* para gerenciar informações.

Ordenação em Tempo Linear

Bibliografia

Bibliografia

Os conteúdos deste material, incluindo figuras, textos e códigos, foram extraídos ou adaptados de:

 Cormen, Thomas H. and Leiserson, Charles E. and Rivest, Ronald L. and Stein, Clifford.

Introduction to Algorithms.

The MIT Press, 2011.

Exercício

Exercício

- ▶ Dada a sequência de números: 3 4 0 2 5 1 3.
- ▶ Ordene em ordem crescente utilizando o algoritmo aprendido em sala (*Heap Sort*), apresentando a sequência dos números a cada passo (Teste de Mesa).